

Pourquoi C++ en 2017 ?



Pourquoi utiliser C++ en 2017 ? La question est posée... C++ est un descendant de C, regardez :

Christophe PICHAUD
Consultant sur les technologies Microsoft
christophepichaud@hotmail.com
www.windowscpp.net

Pour rappel, tous les logiciels que j'utilise sur mon PC portable sont écrits en C/C++ :

- Windows et ses 3400 dlls ;
- Explorer, l'explorateur de fichiers ;
- Le gestionnaire de tâches ;
- MS Paint ;
- Mon lecteur multimédia ;
- Les navigateurs Web (Chrome, Firefox, Internet Explorer, Edge) ;
- La suite Office (Word, Excel, PowerPoint, Outlook, etc) ;
- Notepad++ ;
- Acrobat Reader ;
- Mon vieux MSDN Library 2008 SP1.

La seule exception que j'ai sur mon PC est Visual Studio qui est une application hybride C/C++ .NET COM.

Le PC portable que j'utilise pour écrire cet article est un portable low-cost acheté en 2015 :

- Processeur Intel Celeron Quad Core ;
- Intel HD Graphics ;
- 4 GB RAM ;
- 500 GB Disk.

Et pourtant avec un PC low-cost peu puissant, je peux toujours utiliser en même temps Windows, un browser, Word, MS Paint, Visual Studio et écouter de la musique en arrière-plan. Et je n'ai pas de soucis de performance ; la preuve voici le gestionnaire de tâche : [2]

Bien sûr, ce PC ne fera pas de montage vidéo

ou de la compression diverse et variée... Mais par contre, pour une utilisation standard, il me permet de travailler.

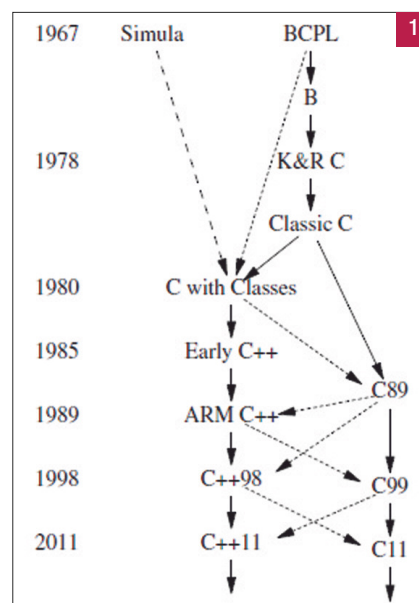
Anatomie des applications

Les applications que je lance consomment peu de mémoire et sont réactives. Pourquoi ? Parce qu'elles utilisent directement les APIs du système d'exploitation qu'est Windows. Les APIs Windows sont utilisables depuis un simple compilateur C/C++. Il suffit de faire `#include <windows.h>` et vous avez accès à des milliers de fonctions. Chaque édition de Visual Studio distribue le SDK Windows. Les applications consomment peu de mémoire et ça c'est la clé de la réussite.

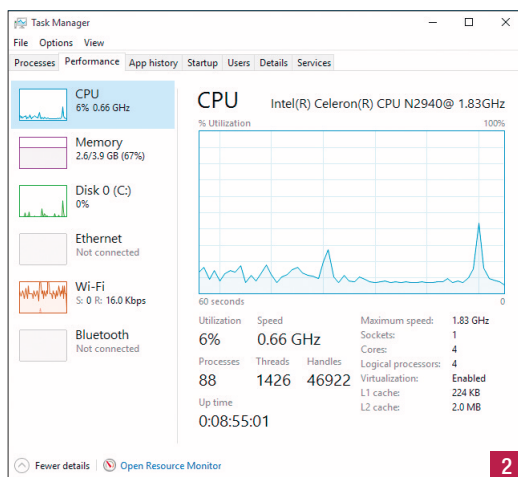
Mythes et réalités

Vous allez me dire, ouaip mais faire une application en C++ *from scratch*, c'est difficile et ce n'est pas productif comme langage le C/C++ hein ? Je réponds de faire attention au discours du marketing et aux préjugés... [3]

En effet, pour développer une application, Microsoft préconise de faire du C#, VB.NET ou JavaScript. Oui mais bon... la question est la suivante : pourquoi Microsoft ne fait pas ses applications en .NET ? On nous avait dit que Microsoft pratiquait le *dog-fooding* : c'est-à-dire que lorsque MS sort une technologie, MS la réutilise dans ses produits. Ah oui, mais c'est un



peu plus complexe que ça. Pour Microsoft, le C++ c'est l'électricité, c'est naturel. Pour le marketing, .NET c'est limpide. Donc, qui croire ? Si vous voulez faire une application qui ne fonctionne que sous Windows, oui .NET peut-être une alternative mais attention, il faut redistribuer le framework .NET. Vous allez me dire oui mais avec .NET Standard 2.0, on peut compiler sous Windows, Mac et Linux. Je vais vous la faire courte... Il y a 10 ans, il existait un package nommé Mono qui permettait de faire du C# sous Linux. Ça a été un flop. Maintenant que Microsoft a racheté la société Xamarin et que .NET Standard 2.0 est sorti, le marketing nous (r)envoie ses sirènes pour faire des applications mobiles et desktop via .NET et C# sur plusieurs plateformes... Méfiez-vous des effets d'annonces car ce n'est pas aussi simple que cela est énoncé. Faites-vous votre propre expérience. En C/C++ sous Windows, il suffit de redistribuer MSVCRedist.exe qui fait 10 MB. Ce package contient la runtime du C, la STL, les DLLs des



2

C++, mythes et réalités

principes et techniques

gestion des erreurs

pointers

Un langage orienté objet compliqué !

Code crado mais je continue sur le même modèle...

cast

Des macros et de goto !

Orienté objet

Complicé à apprendre !

Bas niveau !

Interfaces riches

Peu productif !

Structure de données compactes et efficaces

Non sécurisée, dangereux !

Template meta-programming

C'est du C !

Code dur à maintenir

new / delete

Un bon code

le style C++ 11

3

MFC. De plus, Windows contient déjà plusieurs versions de ces modules.

Pour faire une application de bon standing, il faut être capable de fournir une interface graphique élégante, des librairies qui consomment peu de mémoire et des opérations qui ne poussent pas le processeur dans ses derniers retranchements. Et là, c'est le principe des couches. Si vous développez avec un runtime et un framework, il y aura plusieurs couches à traverser avant que votre code ne rejoigne les APIs du système d'exploitation ou les instructions de votre processeur. La mode est de préserver la batterie, de soulager les disques, et de consommer peu de mémoire. Seul le C/C++ vous permet cela. Au niveau du langage C, on trouve les fondamentaux suivants :

- Gestion de la mémoire avec les fonctions `malloc/free` ;
- Gestion des I/O : `stdio.h` et les fonctions `open`, `close`, `read`, `write`, `fopen`, `fprintf`, `fread`, `fwrite` ;
- Gestion des pointeurs `int *` `ptr` ;
- Etc.

Le C++ fournit un niveau plus abstrait via la STL (Standard Template Library) qui gère :

- La gestion automatique de la mémoire : on évite `new/delete` via `unique_ptr<T>` et `shared_ptr<T>` ;
- Gestion des strings et des buffers avec `string`, `wstring` et les streams ;
- Gestion des containers (`array`, `vector`, `list`, `map`, `set`, etc.) ;
- Des d'algorithmes ;
- Etc.

Au lieu de présenter mon PC portable low-cost, je pourrais vous parler des distributions Linux qui tournent sur de vieilles machines. Pourquoi ? Tout est fait en C/C++ à 99%. Bref, on positionne le curseur sur les éléments suivants :

- Gérer la batterie ;
- Gérer la mémoire ;
- Gérer le processeur.

Avec les langages dit productifs, vous n'avez aucun moyen d'optimiser ces 3 facteurs car vous êtes dans les hautes sphères d'un runtime (CLR ou JRE) qui fait plein de choses tout

seul... Sur un poste de bureau, on peut cacher cela, mais sur un mobile, la sanction est immédiate. Si l'application suçote la batterie ou si une application passe son temps à chauffer la batterie, ce n'est pas bon et l'application n'aura pas un grand succès.

Le C/C++ est portable et natif

Sur chaque système (Windows, Linux, Mac), il existe un compilateur C/C++. Il est ainsi possible de faire du code portable. Bien sûr, le code qui utilise les API Windows ne compilera pas sur Linux mais le code métier, les classes Poco et les libs *third-party* compilent. L'avantage du C/C++ est que le système d'exploitation est fait avec du C. Les autres APIs des librairies *third-party* sont aussi faites en C/C++. Donc quel est cet avantage ? Sous Windows, vous voulez envoyer par exemple des messages cryptés ; rien ne vous interdit de récupérer une librairie de chiffrement sur Linux, de la recompiler, et de l'utiliser sous Windows. Les combinaisons sont infinies.

De plus, les compilateurs C/C++ sont gratuits.

La documentation

Si vous voulez débiter, il existe une ressource terrible : MSDN Library 2008 SP1. Cet ensemble d'articles contient toutes les ressources pour développer sous Windows. On y trouve aussi des articles sur le C/C++ et les MFC.

Téléchargez ce fichier ISO de 2.8 GB depuis le *Microsoft Download Center* et installez-le. [4]

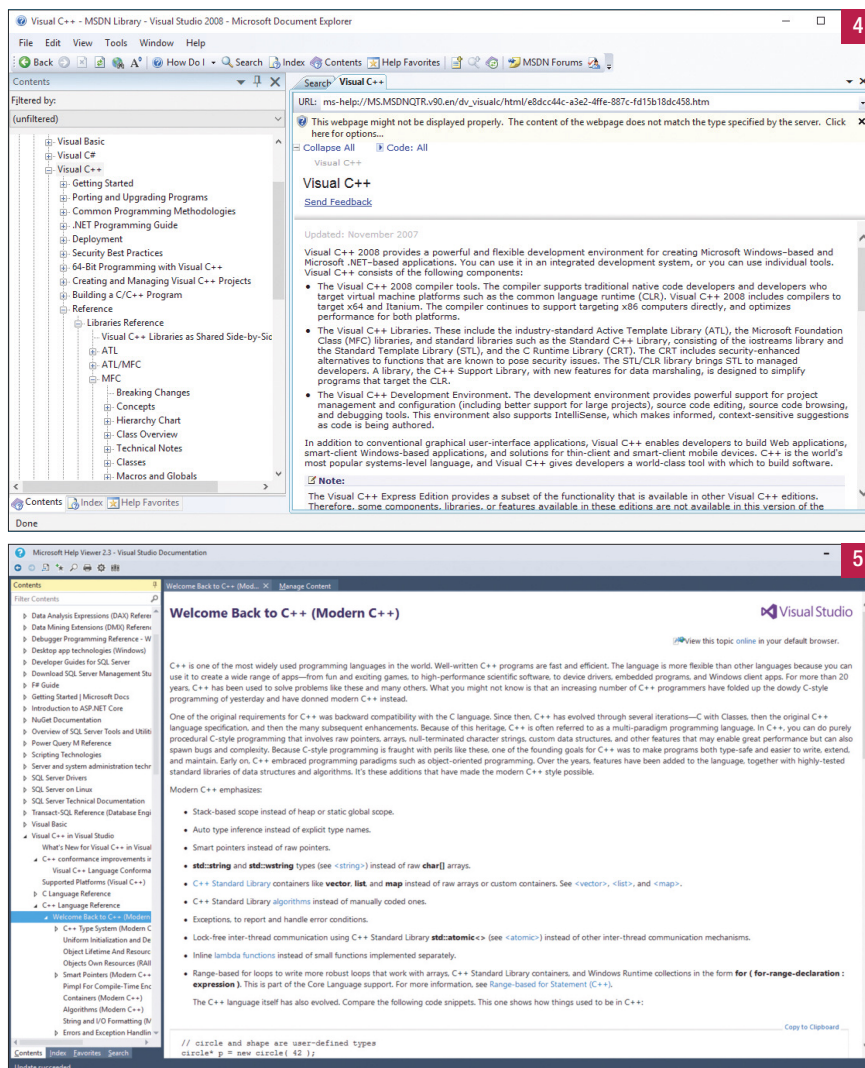
Il existe aussi une version plus récente de la documentation Microsoft. Avec Visual Studio, il existe une application nommée *Help Viewer* et dans laquelle il est possible de télécharger différentes sections de l'aide (.NET, Visual C++, SQL Server, etc). [5]

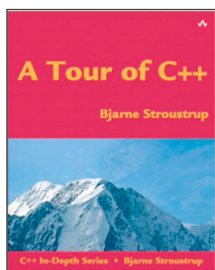
L'IDE

Là, le choix se porte sur Visual Studio. VS existe en version gratuite dite *Community Edition*. Cette version est le produit d'appel vers la version payante de VS comme VS Pro. Le coût du produit est environ 500€.

L'IDE Visual Studio est très confortable mais il faut bien comprendre que ce produit contient moult fonctionnalités. Le problème de Visual Studio est qu'il est de plus en plus lourd et donc il faut savoir patienter si votre machine de développement n'est pas très rapide... Bref, vous êtes prévenu ! [6]

Il est possible aussi d'utiliser VS Code mais ce n'est pas pareil...





Par où commencer ?

Le premier pas est d'apprendre le langage dans sa mouture C++ 11 ou C++ Moderne. Un bon ouvrage, écrit par le créateur du langage, est disponible depuis

2014 : *A Tour of C++* par Bjarne Stroustrup.

La STL (Standard Template Library)

La STL est la bibliothèque du C++. Elle contient de nombreux fichiers d'entêtes. L'évolution du compilateur est indissociable de l'évolution de la STL. La STL fournit des classes templates pour satisfaire tous les besoins principaux : cela va des algorithmes aux containers, des strings aux threads, etc. Voici les fichiers d'entêtes : [7]

Les classes

Il est possible de créer des types concrets ou abstraits. Pour créer une classe abstraite, il suffit de déclarer une méthode virtuelle pure. L'héritage peut être simple ou multiple.

<algorithm>	copy(), find(), sort()
<array>	array
<chrono>	duration, time_point
<cmath>	sqrt(), pow()
<complex>	complex, sqrt(), pow()
<forward_list>	forward_list
<fstream>	fstream, ifstream, ofstream
<future>	future, promise
<ios>	hex, dec, scientific, fixed, defaultfloat
<iostream>	istream, ostream, cin, cout
<map>	map, multimap
<memory>	unique_ptr, shared_ptr, allocator
<random>	default_random_engine, normal_distribution
<regex>	regex, smatch
<string>	string, basic_string
<set>	set, multiset
<sstream>	istringstream, ostringstream
<stdexcept>	length_error, out_of_range, runtime_error
<thread>	thread
<unordered_map>	unordered_map, unordered_multimap
<utility>	move(), swap(), pair
<vector>	vector

Les templates

Un template est une classe qui prend en paramètre un ou plusieurs types. Le template est évalué à la compilation. Il existe aussi des fonctions template.

Le C++ Moderne

Voici les principaux éléments du C++ moderne :

- Utilisation de la stack plutôt que le heap ;
- Utilisation de auto pour masquer le type réel ;
- Utilisation des smart pointers à la place des pointeurs normaux ;
- Utilisation des types std::string ou std::wstring à la place de char[] ;
- Utilisation des containers de la STL vector, list et map à la place des structures customs ;
- Utilisation des algorithmes de la STL à la place des routines customs ;
- Utilisation des exceptions pour remonter des erreurs ;
- Utilisation du type STL std::atomic<> à la place des mécanismes IPC custom ;
- Utilisation des lambdas inline au lieu des petites fonctions déclarées séparément ;
- Utilisation du range for pour le parcours des tableaux et autres containers ;

L'utilisation du mot-clé auto permet de masquer des types complexes à utiliser. Ainsi pour déclarer un itérateur sur un vector de shared_ptr<T>, il ne faut plus écrire :

```
vector<shared_ptr<MyClass>>::const_iterator it = v.begin();
```

A la place on écrit ça :

```
auto it = v.begin();
```

Avec le C++ moderne, il n'y a pas besoin d'utiliser les mécanismes de new/delete pour allouer ou libérer de la mémoire. On utilise, à la place, des smart pointers comme unique_ptr<T> ou

shared_ptr<T> qui savent libérer la mémoire automatiquement.

Le développement multi-plateformes

Revenons sur deux définitions qui sont galvaudées : le natif et le multi-plateformes. Quand on parle de développement natif, on parle de développement qui utilise le même langage que celui qui a permis de faire le système d'exploitation. Concrètement c'est du C/C++. Sous Windows, Linux et Mac, c'est comme ça. Ça veut dire que l'on utilise directement les APIs du système d'exploitation qui sont exploitées dans les entêtes .h fournies par le SDK de l'OS. Pour un programme Windows, cela veut dire qu'il charge kernel32.dll, user32.dll et gdi32.dll. A partir du moment où vous utilisez un langage comme C# qui contient un garbage collector et utilise un runtime, vous n'êtes pas dans le natif. Malheureusement, le marketing raconte des bobards...

Le développement hybride C++ /CLR

Sous Windows, il est possible de faire du code C++ et du code qui appelle des classes .NET via le C++/CLI. Pour faire cela, il faut que le projet supporte l'option *Common Language Runtime Support /clr*. Il est ainsi possible de faire des ajouts de référence à des *assemblies* .NET dans un projet C++. Si vous avez du code .NET, vous pouvez le réutiliser et cela se fait très facilement. L'avantage c'est que si vous avez des besoins XML ou ADO.NET, vous aurez de suite le support du .NET Framework à portée de main.

Framework graphique

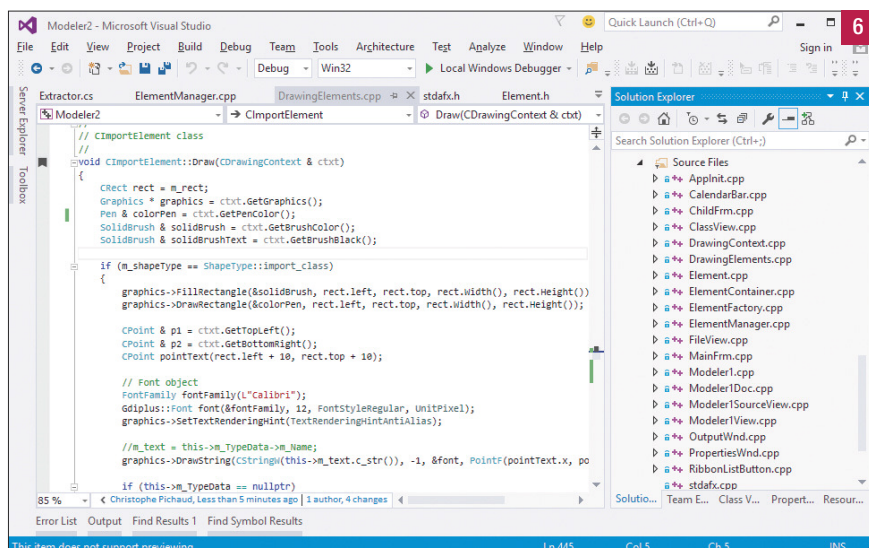
Pour développer une application graphique, il faut avoir des APIs pour afficher des contrôles, des fenêtres, des menus, des boîtes de dialogue, etc. Chaque système (Windows, Linux, Mac) fournit des APIs pour cela. Il existe aussi différents Frameworks pour arriver à faire des applications fenêtrées :

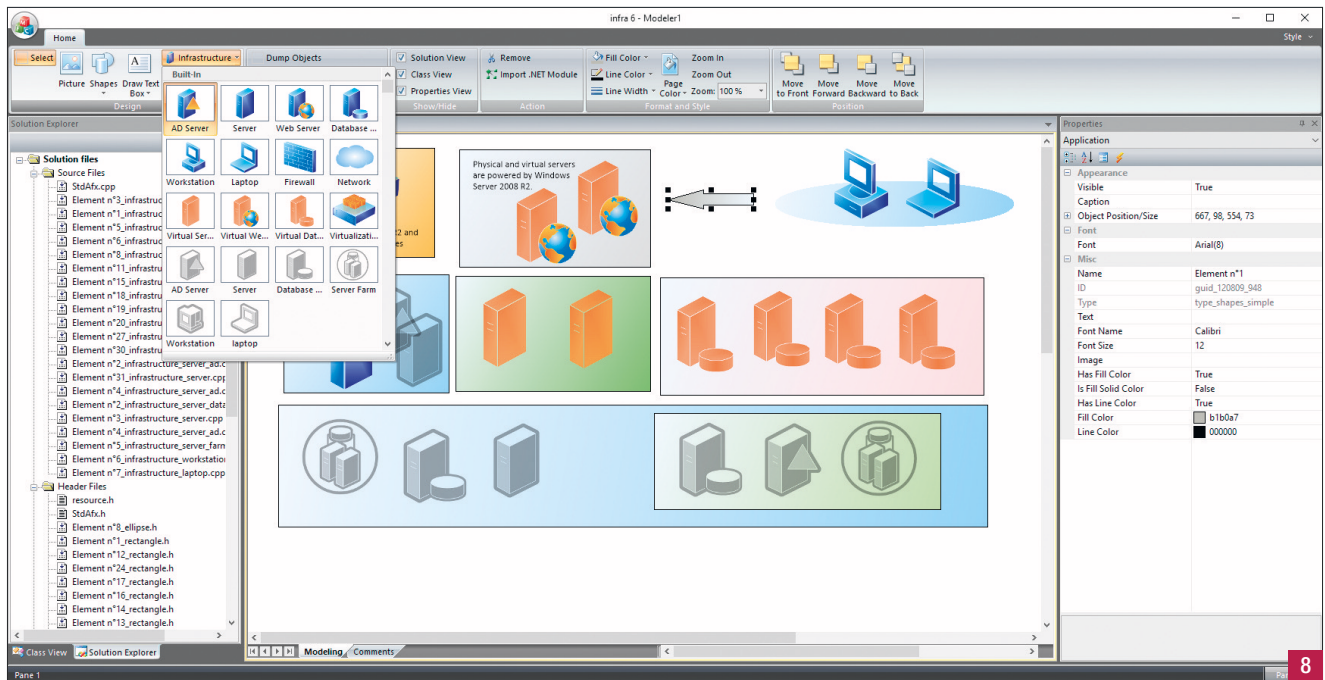
- GTK
- Qt
- MFC
- WTL
- AppKit, Cocoa
- WxWidgets

Sous Windows, il y a 3 options :

- GDI32 : API C historique de gestion des fenêtres ;
- MFC : Microsoft Foundation Classes ;
- WTL : Windows Template Library.

Personnellement, j'ai commencé avec GDI32 pour apprendre les bases. De nos jours, je suis





un adepte des MFC car je suis très productif avec. Le framework fournit les bases des applications modernes à la sortie du Wizard. Avec GDI32, il faut tout coder et cela prend du temps. De plus, MFC fournit des contrôles sophistiqués comme les docking pane, les propriétés grid, le Ribbon, les MDI avec un modèle document-vue ; c'est un peu différent du MVC mais les principes sont les mêmes. Voici le type d'applications que l'on peut faire facilement avec les MFC : [8]
Cette application possède tous les contrôles graphiques qui sont sexy et fonctionnels.

Le Back-end en C++

Il est possible de construire la partie serveur d'une application via plusieurs technologies. Il est possible de construire :

- Un service Windows avec des IPC ;
- Un serveur XML-RPC custom ;
- Un serveur Web API qui retourne du JSON via le REST SDK.

L'avantage du REST SDK, c'est qu'il peut retourner des data au format JSON pour un front qui n'est pas forcément en C++. Le serveur C++ fait avec le REST SDK fera l'objet d'un article séparé.

CONCLUSION

Sous Windows, le compilateur Visual C++ fête ses 25 ans ! Il n'a jamais été autant surpuissant. Le C++ Moderne ouvre de grandes possibilités car le code est plus simple à écrire, plus lisible et plus performant. La STL a de nombreuses fonctionnalités à utiliser comme les containers, les smart pointeurs. Bref, le langage est très abordable même si vous avez des bases de Java ou C# car ceux-ci ne sont que des descendants du C++...

reste cadre de 174 mm de large x 90 mm de haut